모바일 프로그래밍 팀 프로젝트 보고서

[먹뛰]

조장 2022204065 윤성근

2020204080 황승훈

1. 프로젝트 소개 및 개발 배경

① 개발 동기 : 기존 앱은 단순 기록에 그쳐 "얼마나 운동해야 하는가"에 대한 명확한 답을 주지 못해 동기 부여가 어렵다는 한계가 있었습니다.

② 프로젝트 목적 : 식단(칼로리)과 환경 변수(가용 시간)를 분석하여, 현재 상황에서 즉시 실천 가능한 '최적의 운동량'을 제안하는 것을 목표로 합니다.

③ 전략적 접근 : 장비나 장소 제약이 없는 'Running'과 'Walking'을 핵심 솔루션으로 채택, 진입 장벽을 낮춰 지속 가능한 건강 관리를 유도합니다.

2. 개발한 앱의 특별한 점 또는 핵심내용

- 주요 기능 소개

① 식단 입력 및 관리 :

- 기능 : 아침, 점심, 저녁 식단을 입력하면 데이터베이스에 저장된 식품 영양 정보를 바탕으로 총 섭취 칼로리를 자동 계산합니다.
- 특징 및 구현 상세 :
  - 검색 편의성 : AutoCompleteTextView를 통해 DB 내의 수천 가지 음식을 실시간으로 검색할 수 있도록 구현하여 입력 편의성을 극대화했습니다. (※ 상세 구현 방식은 '기술 구현 특이점' 참조)
  - 칼로리 연산:단순히 1인분 기준이 아닌, 사용자가 입력한 섭취량(g)을 변수로 받아 saveSingleMeal메소드 내부에서 (100g당 칼로리 * 입력량 / 100)공식을 적용해 소수점 단위까지 계산합니다.

```
// MealActivity.java : saveSingleMeal()
// DB에서100g당 칼로리(kcalPer100) 조회 후 비례식 적용
double totalKcal = kcalPer100 * (amount / 100.0);
```

  - 데이터 저장 방식 : 식단 데이터는 기존 기록을 덮어쓰는 UPDATE가 아닌, 계속해서 쌓이는 INSERT쿼리를 사용하여 설계했습니다. 따라서 음식 가짓수에 제한 없이 데이터를 축적할 수 있으며, 초기화는 별도의 리셋 기능을 통해서만 수행됩니다.

```
// MealActivity.java : saveSingleMeal()
// 덮어쓰지 않고 새로운 행(Row)으로 추가(누적 구조)
db.execSQL("INSERT INTO meal_log(..., amount_g, calorie) VALUES(?,?,...)", ...);
```

② 체중 입력 및 관리

- 기능 : 정확한 운동 처방을 위해 사용자의 체중을 입력받습니다.
- 특징 및 구현 상세 :
  - 데이터 무결성 : 체중 데이터는 운동 소모량 계산의 핵심 변수이므로, 중복을 방지하고 항상 최신 상태(단일 데이터)를 유지하도록 관리합니다.

```
// WeightActivity.java : btnSave.setOnClickListener
db.execSQL("DELETE FROM user_info"); // 기존 데이터 삭제(중복 방지)
db.execSQL("INSERT INTO user_info(weight) VALUES(?)", ...); // 신규 삽입
```

③ 맞춤형 운동 처방 (핵심 기능)

- 기능 : '오늘 섭취한 칼로리'와 '운동 가능한 시간'을 입력하면, 해당 시간 동안 목표 칼로리를 태우기 위해 필요한 운동 종목(걷기, 달리기 등)과 강도를 분석하여 리포트로 제공합니다.
- 특징 및 구현 상세 :
  - 역산 구조 : 단순히 칼로리만을 보여주는 것이 아니라, 사용자의 입력 시간(minutes)을 변수로 하여 시간당 목표 소모 칼로리를 역산합니다.

```
// ReportActivity.java : recommendExercise()
// 목표치 역산 공식: (총 섭취 칼로리) * (60분/ 입력 시간)
double targetPerHour = totalKcal * (60.0 / minutes);
```

  - 최적 근사치 탐색 : DB 내의 모든 운동 데이터와 위에서 계산한 목표치(targetPerHour) 간의 오차(Math.abs)를 전수 계산하여, 오차가 가장 적은(minDiff) 최적의 운동 강도를 찾아냅니다.

```
// ReportActivity.java : recommendExercise()
for (ActivityRow r : rows) {
double diff = Math.abs(targetPerHour - dbKcalPerHour); // 오차 계산
if (diff < minDiff) { minDiff = diff; best = r; } // 최적값 갱신
}
```

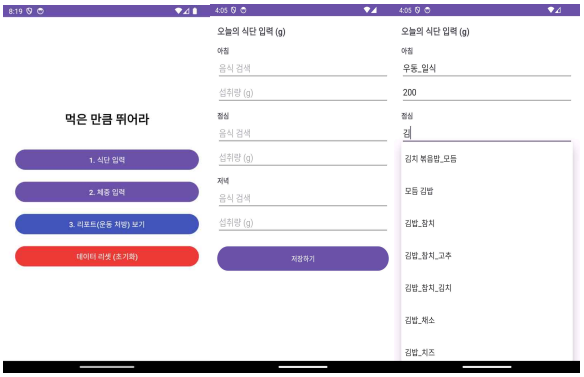
  - 체중 보정 : 운동 데이터는 11kg 단위의 구간으로 나뉘어 있습니다. getWeightBracket메소드에서 사용자 체중을 구간 기준값과 비교하여 가장 근접한 값으로 자동 매핑합니다(정확한 중간값일 경우 반올림 적용).

④ 데이터 초기화 및 관리

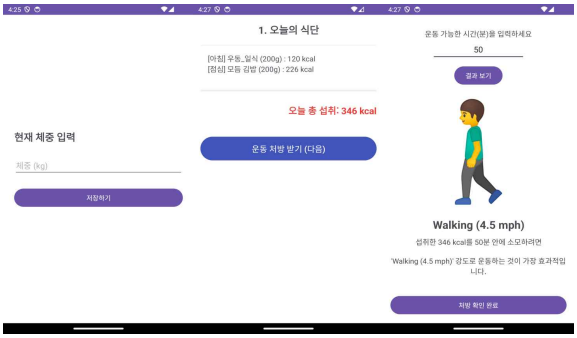
- 기능 : 앱 최초 실행 시 방대한 양의 식품 및 운동 데이터를 자동으로 구축하며, 사용자 요청 시 개인의 기록만 선별적으로 초기화하는 기능을 제공합니다.
- 특징 및 구현 상세 :
  - 중복 실행 방지 플래그 : SharedPreferences를 활용하여 최초 실행 여부를 체크하여 이미 데이터가 준비된 상태라면 로딩 과정을 건너뛰고 즉시 앱이 실행되도록 하여 초기 구동 속도를 단축했습니다.

```
// MainActivity.java : onCreate()
SharedPreferences prefs = getSharedPreferences("prefs", MODE_PRIVATE);
boolean imported = prefs.getBoolean("csv_imported", false);
if (!imported) {
// 최초1회만CSV 파싱 및DB 구축 실행
CSVImporter.importCSV(...);
}
```

▶ 메인 인터페이스 ▶ 식단 입력



▶ 체중 입력 ▶ 리포트 출력



- 데이터베이스 구조 (SQLite Schema)

[DBHelper.java]를 통해 관리되는 내부 데이터베이스는 다음과 같은 4개의 테이블로 구성되어, 정적 데이터와 동적 사용자 데이터를 효율적으로 분리하여 관리합니다.

테이블명	구분	주요 컬럼 및 역할
food_master	기초 데이터	food_name(음식명), kcal(100g당 칼로리), 탄수화물, 단백질, 지방 정보(g)
activity_master	기초 데이터	activity(운동명), intensity(강도), kg48~kg92(체중별 1시간당 소모 칼로리, 11kg 간격)
meal_log	사용자 데이터	date(날짜), meal_type(아침/점심/저녁), food_name, amount_g(섭취량), calorie(계산된 칼로리)
user_info	사용자 데이터	weight(사용자 체중) WeightActivity에서 항상 단일 레코드로 유지

- 기술 특이점

① 대용량 데이터 처리 최적화 (Transaction 적용)

```
// CSVImporter.java
db.beginTransaction(); // 1. 트랜잭션 시작(메모리 버퍼 사용)
```

```
while ((line = br.readLine()) != null) {
// ... 데이터 파싱...
db.execSQL("INSERT INTO ... VALUES(?,?,...)", ...); // 2. 대량 삽입 수행
}

db.setTransactionSuccessful(); // 3. 일괄 커밋
db.endTransaction(); // 4. 트랜잭션 종료
```

앱 실행에 필요한 수천 개의 식품 및 운동 데이터를 건별로 입력하는 방식은 과부하와 속도 저하를 유발하는 원인이었습니다. 이에 대한 해결책으로 DB에 트랜잭션 처리를 적용했습니다. beginTransaction에서 모든 데이터를 하나의 작업 단위로 묶어 메모리에서 우선 처리하고, setTransactionSuccessful로 단 한 번만 저장소에 기록함으로써 입출력(I/O) 부하를 최소화하고 안정적인 구동 환경을 확보했습니다.

② 검색 편의성을 위한 자동완성 구현

```
// MealActivity.java
// 1. DB에서 모든 음식 이름 로드
Cursor c = db.rawQuery("SELECT food_name FROM food_master", null);
ArrayList<String> foods = new ArrayList<>();
while (c.moveToNext()) { foods.add(c.getString(0)); }

// 2. 어댑터 연결(자동완성 활성화)
ArrayAdapter<String> adapter = new ArrayAdapter<>(..., foods);
acBreakfast.setAdapter(adapter);
```

사용자 경험(UX)을 위해 AutoCompleteTextView 위젯을 활용했습니다.
- 안드로이드의 AutoCompleteTextView위젯을 활용하여, 검색어의 일부만 입력해도 DB 내의 유사한 음식명을 실시간으로 추천해 줍니다.
- 앱 구동 시 food_master 테이블의 전체 음식 데이터를 조회하여 메모리(ArrayList)에 적재하고, 이를 검색창 어댑터(ArrayAdapter)에 연결하는 방식으로 지연 시간 없는 즉각적인 검색 환경을 구현했습니다.

③ 입력값 유효성 검사 및 예외 처리

```
// WeightActivity.java : NULL 체크
String s = edtWeight.getText().toString().trim();
if (s.isEmpty()) { // 입력값이 비어있을 경우 방어
Toast.makeText(this, "체중을 입력해주세요.", ...).show();
return; // 로직 중단
}
```

```
// ReportActivity.java : 숫자 변환 예외 처리
try {
double inputMinutes = Double.parseDouble(timeStr);
if (inputMinutes <= 0) { ... } // 0 이하 숫자 입력 방지
// ... 정상 로직 실행...
} catch (NumberFormatException e) { // 숫자가 아닌 문자 입력 시 예외 처리
Toast.makeText(this, "숫자만 입력해주세요.", ...).show();
}
```

사용자 입력 과정에서 발생할 수 있는 빈 값(Null)이나 숫자 형식이 아닌 문자 입력 등의 예

외 상황을 방어하기 위해 try-catch및 조건문(if)을 적용했습니다. 이를 통해 앱의 비정상 종료(Crash)를 미연에 방지하고 데이터 무결성을 확보했습니다.

④ 사용자 피드백 시스템 (Toast Message)

```
// 저장 성공(MealActivity.java):
Toast.makeText(this, "식단이 저장되었습니다.", Toast.LENGTH_SHORT).show();

// 초기화 알림(MainActivity.java):
Toast.makeText(this, "사용자 데이터가 모두 초기화되었습니다.",Toast.LENGTH_SHORT).show();

// 에러 알림(WeightActivity.java):
Toast.makeText(this, "올바른 숫자를 입력하세요.", Toast.LENGTH_SHORT).show();
```

데이터 저장 성공, 초기화 완료, 입력 오류 등 앱의 상태 변화나 사용자의 작업 결과를 직관적으로 인지할 수 있도록 Toast메시지를 활용하여 즉각적인 시각적 피드백을 제공했습니다.

3. 역할 분담

(Data & Logic)

[황승훈]

[핵심 알고리즘 및 데이터베이스 구축]

- DB 설계 : SQLite 데이터베이스 스키마 설계 및 DBHelper클래스 구현 (식단, 운동, 유저 테이블)
- 데이터 처리 : AssetManager를 활용한 CSV 파일 파싱 및 DB 적재 모듈(CSVImporter) 개발
- 로직 구현 : 사용자 입력 시간 기반 운동 추천 알고리즘 로직 작성
- 및 보고서 작성

(프로젝트 기획 / UI & UX)

[윤성근]

[화면 구성 및 액티비티 관리]

- UI 구현 : Linear 및 ScrollView 등 뷰 구성
- 기능 연결 : 각 Activity별 생명주기 관리 및 Intent를 이용한 화면 전환 처리
- 사용자 편의 : AutoCompleteTextView적용 및 입력 값 유효성 검사(예외 처리), Toast 메시지 피드백 구현
- 및 발표, 관련 자료 제작

4. 기대효과 및 활용분야

① 기대효과 : 복잡한 계산 없이 직관적인 운동 지침을 제공하여, 초보자도 쉽게 성취감을 느끼며 운동을 생활화할 수 있습니다.

② 활용분야 : 향후 AI 비전 기술(음식 촬영) 도입을 통해 편의성을 높이고 정교한 맞춤형 헬스케어 플랫폼으로 확장 가능합니다.

③ 한계점 및 개선점 : 일차원적인 체중 기반 데이터를 넘어, BMI, 체성분 등 개인별 신체 변수를 반영하여 알고리즘의 정밀도를 높여 더욱 정확한 리포트를 제공할 수 있습니다.